

Parameterized Constructor

Constructors in Java: With and Without Parameters

In Java, a **constructor** is a special method used to initialize objects. Constructors are invoked when an object of a class is created. Java supports two types of constructors:

1. **Default Constructor** (without parameters)
2. **Parameterized Constructor** (with parameters)

1. Default Constructor (Constructor without Parameters):

A **default constructor** is a constructor that does not take any parameters. If no constructor is provided by the developer, Java automatically provides a default constructor that initializes the object with default values (e.g., `null` for objects, `0` for numeric types).

2. Parameterized Constructor:

A **parameterized constructor** is a constructor that takes one or more arguments. It allows you to initialize an object with specific values at the time of its creation.

Code Example Using Both Constructors:

```
public class Car {
    // Instance variables
    String make;
    String model;
    int year;

    // Default Constructor (Without Parameters)
    public Car() {
        // Initializing fields with default values
        make = "Unknown";
        model = "Unknown";
        year = 0;
    }

    // Parameterized Constructor (With Parameters)
    public Car(String make, String model, int year) {
        // Initializing fields with given values
        this.make = make;
        this.model = model;
        this.year = year;
    }
}
```

```

    }

    // Method to display car details
    public void display() {
        System.out.println("Car make: " + make);
        System.out.println("Car model: " + model);
        System.out.println("Car year: " + year);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object using the default constructor
        Car defaultCar = new Car();
        System.out.println("Default Car Details:");
        defaultCar.display(); // Should print default values

        System.out.println();

        // Creating an object using the parameterized constructor
        Car customCar = new Car("Toyota", "Corolla", 2021);
        System.out.println("Custom Car Details:");
        customCar.display(); // Should print the custom values
    }
}

```

Explanation:

1. Default Constructor:

- The `Car()` constructor does not take any parameters and initializes the object's fields with default values: "Unknown" for `make` and `model`, and `0` for `year`.

2. Parameterized Constructor:

- The `Car(String make, String model, int year)` constructor takes three parameters (`make`, `model`, and `year`) and initializes the object with the provided values.

3. Displaying the Object:

- The `display()` method is used to print out the current values of the instance variables.

4. In the `Main` class:

- The first object (`defaultCar`) is created using the default constructor, which sets the fields to the default values. The `display()` method is called to print these values.
- The second object (`customCar`) is created using the parameterized constructor, which sets the fields to the values "Toyota", "Corolla", and 2021. The `display()` method is called to print these values.

Output:

Default Car Details:

Car make: Unknown

Car model: Unknown

Car year: 0

Custom Car Details:

Car make: Toyota

Car model: Corolla

Car year: 2021

Key Points:

1. Default Constructor:

- Initializes the object with default values.
- If no constructor is explicitly defined, Java automatically provides a default constructor.

2. Parameterized Constructor:

- Allows you to initialize the object with specific values at the time of object creation.
- You can define multiple constructors with different parameters (constructor overloading) to handle different initialization scenarios.